

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)** Department of Computer Science and
Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA0620	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	K.Deepika	Reg. No.:	192371042
Section / Batch:	CSE 4th year	Date:	23/07/2026

Instructions to Students

- Answer the following question. It carries 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions
Application-Based Questions	<i>Rank Comparison System</i> A university ranking system compares each student's performance with every other student to determine overall rankings using a pairwise comparison approach. Explain how nested comparisons influence the time complexity of the algorithm and describe the growth rate as the number of students increases. Determine the asymptotic time complexity using Big-O notation and justify your reasoning. Additionally, analyze the space complexity of the system, considering storage of intermediate comparisons. Interpret how this approach impacts performance for large datasets and discuss its efficiency.	Q1

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I K.Deepika(192371042) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: k.deepika

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%				
Application of Concepts	30%				
Problem-Solving Approach	20%				
Accuracy of Solution	20%				

Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Source code:

```
import random
import time
```

```
def run_pairwise_ranking(num_students):
    # 1. Generate student data: (student_id, gpa, research_score)
    students = [
        {
            "id": i,
            "gpa": random.uniform(2.0, 4.0),
            "research": random.uniform(50.0, 100.0),
            "wins": 0
        }
        for i in range(num_students)
    ]

    # 2. Start nested comparison O(n^2)
    start_time = time.perf_counter()
    total_comparisons = 0

    for i in range(num_students):
        for j in range(i + 1, num_students):
            s1 = students[i]
            s2 = students[j]

            # Calculate composite scores
            score1 = (s1["gpa"] * 0.7) + (s1["research"] * 0.3)
            score2 = (s2["gpa"] * 0.7) + (s2["research"] * 0.3)

            # Assign win
```

```

        if score1 >= score2:
            s1["wins"] += 1
        else:
            s2["wins"] += 1

    total_comparisons += 1

execution_time = time.perf_counter() - start_time
return total_comparisons, execution_time

# Execute test runs
if __name__ == "__main__":
    test_sizes = [500, 1000, 2000, 4000]

    print(f'{"Students (n)":<15} | {"Comparisons":<15} | {"Execution Time (s)":<20}')
    print("-" * 55)

    for n in test_sizes:
        comparisons, elapsed = run_pairwise_ranking(n)
        print(f'{n:<15} | {comparisons:<15,} | {elapsed:<20.5f}')

```

OUTPUT:

```

import random
import time

def run_pairwise_ranking(num_students):
    # 1. Generate student data: (student_id, gpa, research_score)
    students = [
        {
            "id": i,
            "gpa": random.uniform(2.0, 4.0),
            "research": random.uniform(50.0, 100.0),
            "wins": 0
        }
        for i in range(num_students)
    ]

```

Students (n)	Comparisons	Execution Time (s)
500	124,750	0.05607
1000	499,500	0.19252
2000	1,999,000	0.76953
4000	7,998,000	3.78961

```

=== Code Execution Successful ===

```